BRACT's

Vishwakarma Institute of Information Technology, Pune Practical Implementation Sheet

Department: CS-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No:6 Roll no: 21 PRN No: 12411682

Subject Name: Deep Learning

Name of Student: Pranay Zunzunwala

Problem Statement: Design and implement a Convolutional Neural Network (CNN) for image classification using the Tom

Algorithm:

Step 1: Import the required libraries such as NumPy, Pandas, Matplotlib, TensorFlow/Keras, YFinance, and the necessary modules from scikit-learn.

Step 2: Load the **Apple (AAPL) stock dataset** using Yahoo Finance for the specified date range and inspect the dataset by displaying its shape, columns, and sample records.

Step 3: Perform **data preprocessing** by handling missing values, selecting the required stock-price feature, and arranging the data in chronological order.

Step 4: Normalize the selected stock-price data using **MinMaxScaler** to scale the values to a suitable range for training the neural network.

Step 5: Create **time-series sequences** using a fixed number of previous observations as input and the next stock-price value as the target.

Step 6: Split the generated sequences into **training and testing datasets** while maintaining the chronological order of the stock data.

Step 7: Build an **LSTM-based Sequential Neural Network** using LSTM, Dropout, and Dense layers, and compile the model using the Adam optimizer and an appropriate loss function.

Step 8: Train the LSTM model on the training data using a specified number of **epochs and batch size**, while using early stopping to prevent overfitting.

Step 9: Use the trained model to **predict stock prices** for the testing dataset and evaluate its performance using **Mean Absolute Error (MAE)** and **Mean Squared Error (MSE/RMSE)**.

Step 10: Visualize the results by plotting the **actual vs. predicted AAPL stock prices** and analyze the model's prediction performance.

Code:

```
# Step 1: Install KerasTuner

!pip install -q keras-tuner

# Step 2: Import Required Libraries

import os

import zipfile

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import tensorflow as tf

import keras_tuner as kt

from google.colab import files

from tensorflow.keras import layers, models

from sklearn.metrics import classification_report, confusion_matrix

print("TensorFlow:", tf.__version__)

print("Keras Tuner:", kt.__version__)

# Step 3: Download Apple Disease Dataset

!wget -q -O apple-disease.zip \

"https://www.kaggle.com/api/v1/datasets/download/rm1000/apple-disease-dataset-original"

print("Dataset downloaded successfully!")
```

```
print(
    "ZIP size:",
    os.path.getsize("apple-disease.zip") / (1024 * 1024),
    "MB"
)
# Step 4: Extract Dataset
!rm -rf /content/apple_dataset
!mkdir -p /content/apple_dataset
!unzip -q apple-disease.zip -d /content/apple_dataset
print("Dataset extracted successfully!")
# Step 5: Inspect Dataset Structure
for root, dirs, file_list in os.walk("/content/apple_dataset"):
    level = root.replace(
        "/content/apple_dataset", ""
    ).count(os.sep)
    if level <= 2:
        print(root)
        for d in dirs:
            print("   ", d)
# Step 6: Locate Apple Dataset Folder
EXPECTED_CLASSES = {
    "Apple___Apple_scab",
    "Apple___Black_rot",
    "Apple___Cedar_apple_rust",
```

```
    "Apple___healthy"
}
DATASET_PATH = None
for root, dirs, file_list in os.walk("/content/apple_dataset"):
    if EXPECTED_CLASSES.issubset(set(dirs)):
        DATASET_PATH = root
        break
if DATASET_PATH is None:
    raise FileNotFoundError(
        "Could not automatically find the four Apple "
        "class folders. Check the dataset structure."
    )
print("Dataset path:", DATASET_PATH)
print("Classes found:", os.listdir(DATASET_PATH))
# Step 7: Count Images in Each Class
for class_name in sorted(EXPECTED_CLASSES):
    class_path = os.path.join(
        DATASET_PATH,
        class_name
    )
    count = sum(
        f.lower().endswith(
            (".jpg", ".jpeg", ".png", ".bmp", ".webp")
        )
```

```
        for f in os.listdir(class_path)
    )
    print(
        f"{class_name}: {count} images"
    )
# Step 8: Load Training and Validation Dataset
IMG_SIZE = (128, 128)
BATCH_SIZE = 32
SEED = 42
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_PATH,
    validation_split=0.20,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_PATH,
    validation_split=0.20,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
```

```
)
class_names = train_ds.class_names
num_classes = len(class_names)
print("Class names:", class_names)
print("Number of classes:", num_classes)
# Step 9: Display Sample Images
plt.figure(figsize=(10, 8))
for images, labels in train_ds.take(1):
    for i in range(min(9, len(images))):
        ax = plt.subplot(
            3, 3, i + 1
        )
        plt.imshow(
            images[i].numpy().astype("uint8")
        )
        plt.title(
            class_names[labels[i]]
        )
        plt.axis("off")
plt.tight_layout()
plt.show()
# Step 10: Apply TensorFlow AUTOTUNE
AUTOTUNE = tf.data.AUTOTUNE
train_ds = (
```

```
    train_ds
    .cache()
    .shuffle(1000)
    .prefetch(AUTOTUNE)
)
val_ds = (
    val_ds
    .cache()
    .prefetch(AUTOTUNE)
)
print("tf.data.AUTOTUNE enabled successfully!")
# Step 11: Data Augmentation
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip(
        "horizontal"
    ),
    layers.RandomRotation(
        0.15
    ),

    layers.RandomZoom(
        0.10
    )
])
```

```
print("Data augmentation ready.")
# Step 12: Build CNN Model
def build_model(hp):
    model = models.Sequential([
        # Input layer
        layers.Input(
            shape=(128, 128, 3)
        ),
        # Data augmentation
        data_augmentation,
        # Normalize pixel values
        layers.Rescaling(
            1.0 / 255
        ),
        # First Convolution Block
        layers.Conv2D(
            filters=hp.Choice(
                "filters_1",
                [32, 64]
            ),
            kernel_size=(3, 3),
            activation="relu",
            padding="same"
        ),
```

```
layers.MaxPooling2D(),
# Second Convolution Block
layers.Conv2D(
    filters=hp.Choice(
        "filters_2",
        [64, 128]
    ),
    kernel_size=(3, 3),
    activation="relu",
    padding="same"
),
layers.MaxPooling2D(),
# Third Convolution Block
layers.Conv2D(
    filters=hp.Choice(
        "filters_3",
        [128, 256]
    ),
    kernel_size=(3, 3),
    activation="relu",
    padding="same"
),
layers.MaxPooling2D(),
# Flatten layer
```

```
    layers.Flatten(),
    # Dense layer
    layers.Dense(
        units=hp.Choice(
            "dense_units",
            [128, 256, 512]
        ),
        activation="relu"
    ),
    # Dropout layer
    layers.Dropout(
        hp.Float(
            "dropout",
            0.2,
            0.5,
            step=0.1
        )
    ),
    # Output layer
    layers.Dense(
        num_classes,
        activation="softmax"
    )
])
```

```
    # Learning rate
    learning_rate = hp.Choice(
        "learning_rate",
        [
            0.001,
            0.0005,
            0.0001
        ]
    )
    # Compile model
    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=learning_rate
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
# Step 13: Create Hyperband Tuner
tuner = kt.Hyperband(
    build_model,
    objective="val_accuracy",
    max_epochs=10,
    factor=3,
```

```
    directory="/content/tuner_results",
    project_name="apple_disease",
    overwrite=True
)
print("Hyperband tuner created successfully!")
# Step 14: Perform Automatic Hyperparameter Tuning
tuner.search(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
# Step 15: Display Best Hyperparameters
best_hps = tuner.get_best_hyperparameters(
    num_trials=1
)[0]
print("=" * 50)
print("BEST HYPERPARAMETERS")
print("=" * 50)
print(
    "Conv1 filters :",
    best_hps.get("filters_1")
)
print(
    "Conv2 filters :",
```

```
    best_hps.get("filters_2")
)
print(
    "Conv3 filters :",
    best_hps.get("filters_3")
)
print(
    "Dense units   :",
    best_hps.get("dense_units")
)
print(
    "Dropout       :",
    best_hps.get("dropout")
)
print(
    "Learning rate :",
    best_hps.get("learning_rate")
)
# Step 16: Get Best CNN Model
best_model = tuner.get_best_models(
    num_models=1
)[0]
best_model.summary()
# Step 17: Train Best CNN Model
```

```
history = best_model.fit(
    train_ds,


    validation_data=val_ds,
    epochs=15
)
# Step 18: Evaluate Model
loss, accuracy = best_model.evaluate(
    val_ds
)
print("=" * 50)
print("MODEL PERFORMANCE")
print("=" * 50)
print(
    f"Validation Loss: {loss:.4f}"
)
print(
    f"Validation Accuracy: "
    f"{accuracy:.4f} "
    f"({accuracy * 100:.2f}%)"
)
# Step 19: Plot Training and Validation Accuracy
plt.figure(figsize=(8, 5))
plt.plot(
```

```
    history.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title(
    "Apple Disease CNN Accuracy"
)
plt.legend()
plt.grid()
plt.show()
# Step 20: Plot Training and Validation Loss
plt.figure(figsize=(8, 5))
plt.plot(
    history.history["loss"],
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
```

```
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title(
    "Apple Disease CNN Loss"
)
plt.legend()
plt.grid()
plt.show()
# Step 21: Classification Report
y_true = []
y_pred = []
for images, labels in val_ds:
    predictions = best_model.predict(
        images,
        verbose=0
    )
    predicted_labels = np.argmax(
        predictions,
        axis=1
    )
    y_true.extend(
        labels.numpy()
    )
```

```
    y_pred.extend(
        predicted_labels
    )
y_true = np.array(y_true)
y_pred = np.array(y_pred)
print(
    classification_report(
        y_true,
        y_pred,
        target_names=class_names
    )
)
# Step 22: Confusion Matrix
cm = confusion_matrix(
    y_true,
    y_pred
)
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=class_names,
    yticklabels=class_names
```

```
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(
    "Apple Disease Confusion Matrix"
)
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
# Step 23: Save the Trained Model
MODEL_PATH = (
    "/content/apple_disease_cnn.keras"
)
best_model.save(
    MODEL_PATH
)
print(
    "Model saved to:",
    MODEL_PATH
)
# Step 24: Upload an Apple Leaf Image
uploaded = files.upload()
if not uploaded:
```

```
    raise ValueError(
        "No image was uploaded."
    )
filename = list(
    uploaded.keys()
)[0]
print(
    "Uploaded:",
    filename
)
# Step 25: Predict Apple Plant Disease
disease_names = {
    "Apple___Apple_scab":
        "Apple Scab",
    "Apple___Black_rot":
        "Apple Black Rot",
    "Apple___Cedar_apple_rust":
        "Apple Cedar Apple Rust",
    "Apple___healthy":
        "Healthy Apple Leaf"
}
# Load image
img = tf.keras.utils.load_img(
    filename,
```

```
    target_size=IMG_SIZE
)
# Convert image to array
img_array = tf.keras.utils.img_to_array(
    img
)
# Add batch dimension
img_array = tf.expand_dims(
    img_array,
    axis=0
)
# Predict
predictions = best_model.predict(
    img_array,
    verbose=0
)
# Get predicted class
predicted_index = np.argmax
    predictions[0]
)
predicted_class = class_names[
    predicted_index
]
# Calculate confidence
```

```
confidence = float(
    np.max(
        predictions[0]
    ) * 100
)
# Convert class name to readable name
predicted_disease = disease_names.get(
    predicted_class,
    predicted_class
)
# Display prediction
print("=" * 50)
print("APPLE DISEASE PREDICTION")
print("=" * 50)
print(
    "Disease    :",
    predicted_disease
)
print(
    f"Confidence : "
    f"{confidence:.2f}%"
)
print("=" * 50)
# Display uploaded image
```

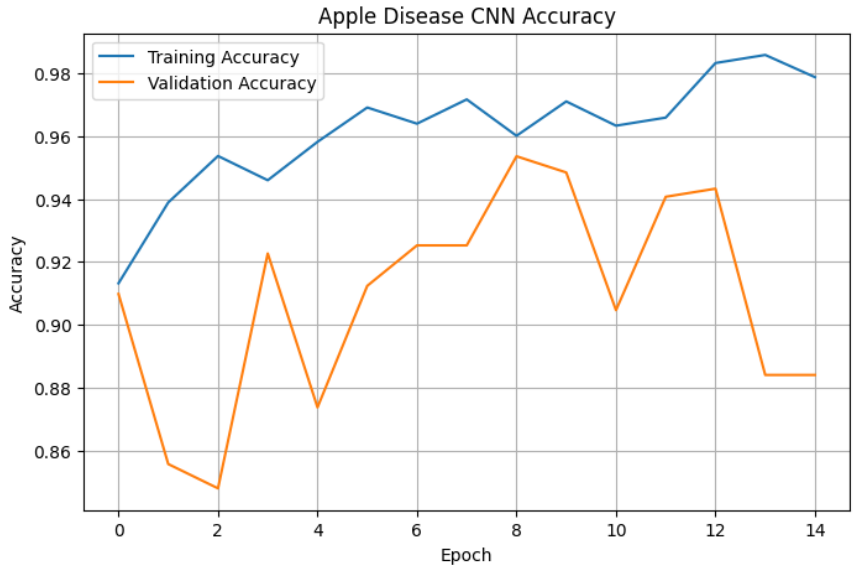
```
plt.figure(figsize=(7, 7))
plt.imshow(img)
plt.axis("off")
plt.title(
    f"{predicted_disease}\n"
    f"Confidence: {confidence:.2f}%"
)
plt.show()
# Display Probability for All Classes
print(
    "Prediction probabilities:"
)
for class_name, probability in zip(
    class_names,
    predictions[0]
):
    readable_name = disease_names.get(
        class_name,
        class_name
    )
    print(
        f"{readable_name:25s}: "
        f"{probability * 100:.2f}%"
    )
```

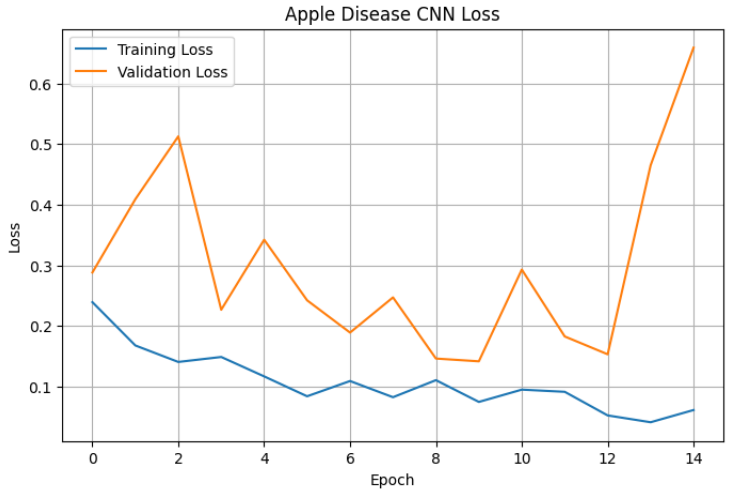
Output:

Model: "sequential"

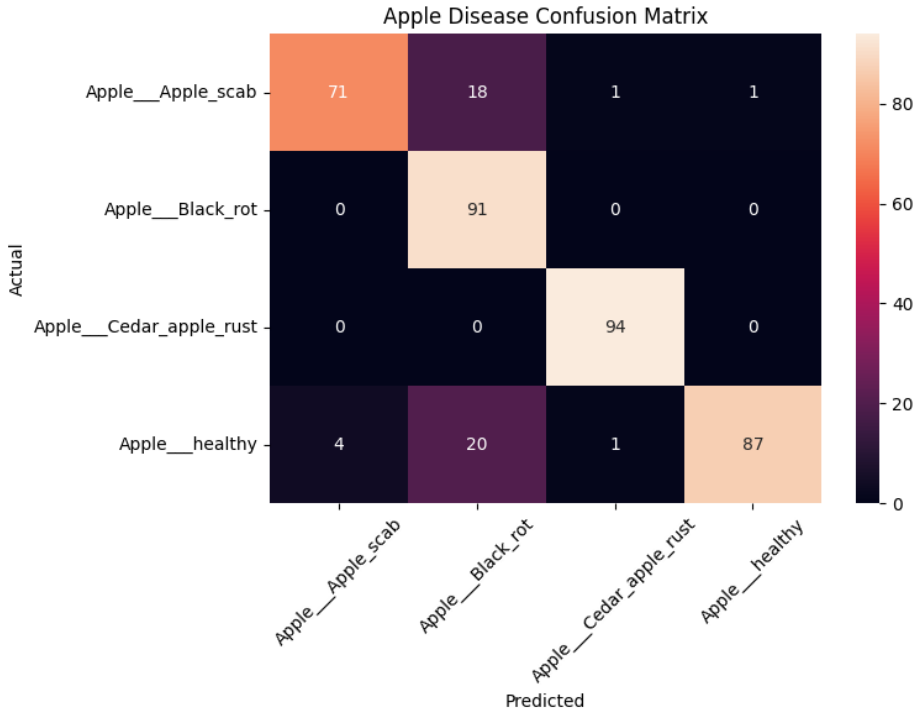
Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 128, 128, 3)	0
rescaling (Rescaling)	(None, 128, 128, 3)	0
conv2d (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_1 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_2 (Conv2D)	(None, 32, 32, 256)	147,712
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 256)	0
flatten (Flatten)	(None, 65536)	0
dense (Dense)	(None, 512)	33,554,944
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 4)	2,052

Total params: 33,724,100 (128.65 MB)
Trainable params: 33,724,100 (128.65 MB)
Non-trainable params: 0 (0.00 B)





	precision	recall	f1-score	support
Apple___Apple_scab	0.95	0.78	0.86	91
Apple___Black_rot	0.71	1.00	0.83	91
Apple___Cedar_apple_rust	0.98	1.00	0.99	94
Apple___healthy	0.99	0.78	0.87	112
accuracy			0.88	388
macro avg	0.90	0.89	0.89	388
weighted avg	0.91	0.88	0.89	388



```
================================================
APPLE DISEASE PREDICTION
================================================
Disease    : Apple Black Rot
Confidence : 100.00%
================================================
```




```
Prediction probabilities:
Apple Scab                : 0.00%
Apple Black Rot           : 100.00%
Apple Cedar Apple Rust    : 0.00%
Healthy Apple Leaf        : 0.00%
```